Formal Reasoning for Intelligent QA Systems: A Case Study in the Educational Domain

Tuan Bui
tuanbc88@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, VietNam

An Nguyen
an.nguyen2452023@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, Vietnam

Phat Thai
phat.thaiquang2004@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, Vietnam

Minh Hua
minh.hua2006@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, Vietnam

Ngan Pham L.N.
ngan.pham1409207@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, Vietnam

Ngan Pham T.B.
ngan.pham11@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, Vietnam

Dung Le
dung.lengochung@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, Vietnam

Long Nguyen
long.nguyencse2023@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, VietNam

Thanh-Tung Tran
tttung@hcmiu.edu.vn
International University - VNU-HCM
Ho Chi Minh City, Vietnam

Thang Bui
bhthang@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, Vietnam

Tho Quan
qttho@hcmut.edu.vn
Ho Chi Minh City University of Technology (HCMUT) - VNU-HCM
Ho Chi Minh City, Vietnam

Abstract

Reasoning is essential for closed-domain QA systems in which procedural correctness and policy compliance are critical. While large language models (LLMs) have shown strong performance on many reasoning tasks, recent work reveals that their reasoning traces are often unfaithful—serving more as plausible justifications than as causally grounded derivations. Efforts to combine LLMs with symbolic engines (e.g., Prover9, Z3) have improved reliability but remain limited to static forms of logic, struggling with dynamic, state-based reasoning such as multi-step progressions and conditional transitions.

In this paper, we propose **MCFR** (Model Checking for Formal Reasoning), a neuro-symbolic framework that integrates LLMs with model checking to support property verification. MCFR translates natural language into formal specifications and verifies them over transition models. To support evaluation, we introduce **EduMC-QA**, a benchmark dataset grounded in real academic procedures. Our results show that MCFR improves reasoning faithfulness and interpretability, offering a viable path toward verifiable QA in high-stakes closed-domain applications. In addition to evaluating MCFR, we compare its performance with state-of-the-art LLMs such as ChatGPT, DeepSeek, and Claude to contextualize its effectiveness.

CCS Concepts

• **Computing methodologies** → **Model verification and validation**; **Temporal reasoning**; **Logic programming and answer set programming**; *Reasoning about belief and knowledge*.

Keywords

Formal Reasoning, Faithful Reasoning, Model Checking, Large Language Models, Intelligent QA Systems

ACM Reference Format:
Tuan Bui, An Nguyen, Phat Thai, Minh Hua, Ngan Pham L.N., Ngan Pham T.B., Dung Le, Long Nguyen, Thanh-Tung Tran, Thang Bui, and Tho Quan. 2025. Formal Reasoning for Intelligent QA Systems: A Case Study in the Educational Domain. In *Proceedings of the 2nd ACM Workshop in AI-powered Question & Answering Systems (AIQAM '25), October 27–28, 2025, Dublin, Ireland*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3746274.3760392

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AIQAM '25, Dublin, Ireland

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-2056-7/2025/10
https://doi.org/10.1145/3746274.3760392

1 Introduction

Question Answering (QA) has emerged as a central task in natural language understanding (NLU) due to its wide applicability in domains such as education, healthcare, and human-computer interaction. A key aspect of QA lies in its inherent requirement

for reasoning—drawing logical conclusions from given premises or background knowledge. While traditional information retrieval-based QA systems focus primarily on pattern matching or keyword alignment, modern intelligent QA systems aim to reason about input questions using deeper semantic understanding. This work is presented as part of the AIQAM'25 workshop on AI-powered Question Answering Systems for Multimedia, which highlights emerging approaches in intelligent QA across domains and modalities [6].

Recent advances in large language models (LLMs) have significantly enhanced QA performance, particularly in tasks involving mathematical or symbolic reasoning. Techniques like Chain-of-Thought (CoT) prompting [3] and Multi-Agent Reasoning [9] simulate human-style step-by-step thinking and have achieved impressive results. However, growing evidence indicates that LLMs often fail at *faithful reasoning*—i.e., their generated reasoning traces do not causally influence the final answer [7, 11]. These models tend to produce post hoc rationalizations rather than genuinely grounded inferences, exposing a gap between surface-level coherence and internal logical validity. To close this gap, neuro-symbolic (NeSy) systems offer a promising hybrid approach by combining the representational power of LLMs with the rigor of symbolic solvers. In this paradigm, LLMs translate NL inputs into formal representations (e.g., first-order logic), which are then processed by engines like Z3 or Prover9 [10].

Example 1. Questions on Student Lifecycle.
Context. The academic progression of a student typically involves multiple stages, such as enrollment, advancement through academic years (Year 1–4), participation in courses and social activities, taking temporary leave, and ultimately reaching either graduation or program withdrawal. Certain actions, such as course registration, are constrained by institutional rules (e.g., prerequisite requirements), while others are subject to timing and eligibility conditions.
Questions. The following questions illustrate four core verification properties that are critical in ensuring the correctness of such educational processes:
◇ **Q1 (Liveness).** Can a student graduate within eight semesters if all requirements are satisfied on time?
◇ **Q2 (Safety).** Is it possible to register for CourseX without passing its prerequisite CourseY?
◇ **Q3 (Reachability).** Can a student advance to Year 3 within three semesters?
◇ **Q4 (Fairness).** Could a student accumulate 100 credits yet fail to advance to Year 3 owing to missing social activity points?

However, symbolic solvers are typically limited to static logic tasks and struggle with dynamic, state-dependent reasoning—common in domains like education, where actions such as course registration or academic progression depend on temporal constraints and evolving eligibility. As illustrated in Example 1, questions involving safety, liveness, reachability, and fairness require reasoning over sequential transitions and conditional policies—posing challenges for symbolic solvers due to limited procedural expressiveness, and for LLMs due to a lack of verifiable grounding.

For instance, Q1 (Liveness) asks whether a student can graduate within eight semesters if all requirements are met on time, requiring that a "good" state is eventually reachable. In contrast, Q2 (Safety) checks whether the system prevents invalid paths, such as registering for a course without completing its prerequisite. Q3 (Reachability) concerns whether an accelerated academic path is viable—for example, if a student can reach Year 3 in just three semesters by fulfilling requirements early. Q4 (Fairness) explores whether non-academic factors, like missing social activity points, could unfairly block academic advancement despite strong academic performance. Both cases underscore the need for nuanced temporal and policy-aware reasoning, which remains a challenge for current methods.

To address these limitations, we introduce **MCFR** (Model Checking for Formal Reasoning), a hybrid framework that augments QA systems with verifiable inference capabilities. MCFR employs LLMs to generate formal representations from NL input, which are then verified using model checking (MC) over full state-transition systems. This allows the system to capture procedural behaviors and validate complex properties such as reachability or fairness in academic workflows. To facilitate systematic evaluation, we also construct a new dataset grounded in real-world educational processes that require both symbolic abstraction and temporal reasoning.
Our contributions are as follows.

- We propose **MCFR**, a formal reasoning framework for QA that integrates LLM-based semantic translation with model checking for verifying temporal and procedural properties.
- We implement a QA prototype grounded in real academic scenarios at Ho Chi Minh City University of Technology (HCMUT), demonstrating the feasibility of state-based formal verification in educational domains.
- We construct a novel QA dataset, **EduMC-QA**, based on real institutional processes at HCMUT, and use it to perform comprehensive evaluations assessing the reasoning capability, coverage, and faithfulness of MCFR. We also compare MCFR's performance against state-of-the-art LLMs such as ChatGPT[1], Claude[2], and DeepSeek[3] to contextualize its effectiveness.

2 Related Works

2.1 Reasoning Patterns of LLMs

LLMs have demonstrated strong performance across various reasoning tasks, particularly through prompting-based techniques such as CoT prompting [3], along with various enhancements that elicit intermediate reasoning steps and improve answer accuracy [4]. Recent advances also explore multi-agent reasoning setups, in which one LLM generates the reasoning steps while another acts as a verifier, identifying errors and prompting the first model to revise its reasoning accordingly [9].

Despite these improvements, concerns remain about the faithfulness of LLM-generated reasoning. Studies have shown that models often produce coherent reasoning traces that are not causally tied to their final answers. For example, [11] inserted misleading or biased cues into reasoning chains and found that LLMs often relied on these biases rather than engaging in true inference. Similarly,

[1] https://chatgpt.com
[2] https://claude.ai
[3] https://chat.deepseek.com

[8] manipulated reasoning steps—by removing key inferences or injecting noise—and observed that LLM predictions remained mostly unchanged. These findings suggest that current LLMs frequently treat reasoning traces as post hoc justifications rather than integral components of decision-making, raising critical questions about their reliability in high-stakes applications.

2.2 Formal Reasoning

Recent work has explored integrating LLMs with symbolic solvers to achieve more faithful and verifiable reasoning. Logic-LM [10] decomposes the reasoning process into three phases: problem formulation, symbolic inference, and interpretation—showing improvements over traditional CoT-style prompting. Other approaches such as SpecGen [5] use mutation-based corrections and verifier feedback to iteratively refine LLM-generated formal specifications. Logic-guided theorem proving [10] further scaffolds LLM outputs with formal logic to reduce semantic errors. In practical applications, LLMs have also been combined with SMT solvers for task planning [2], where the solver verifies constraints and offers corrective feedback during planning failures.

Despite these advances, symbolic solvers like Z3 or Prover9 remain inherently static: they require precise and well-formed logical inputs and struggle with ambiguity, incomplete specifications, or procedural reasoning involving state changes over time. This rigidity makes them fragile in QA pipelines where LLM outputs may be noisy or underspecified. Furthermore, such solvers lack mechanisms to reason over dynamic transitions or evolving states—common in domains like education—limiting their standalone applicability without iterative repair or hybrid integration.

These limitations motivate the development of hybrid frameworks like MCFR, which combine LLM-based translation with state-based formal verification techniques such as model checking.

3 Background

3.1 Neural Symbolic AI

NeSy systems aim to combine the strengths of neural networks—such as perception and natural language understanding—with the formal rigor and interpretability of symbolic reasoning. A common architecture involves using neural models (e.g., LLMs) to translate natural language inputs into formal representations like first-order logic, which are then passed to symbolic engines (e.g., theorem provers or model checkers) for inference. This modular design allows LLMs to handle linguistic ambiguity while relying on symbolic solvers for precise, verifiable reasoning [7].

Recent surveys, such as [12], have categorized NeSy systems into various types based on the level of integration between neural and symbolic components. In many practical QA applications, loosely coupled hybrids—where LLMs act as semantic translators and symbolic modules verify logical constraints—have shown promising results.

3.2 Model Checking

Model checking [1] is an automated formal verification technique used to determine whether a finite-state system model satisfies a given logical specification. The system is typically modeled as a transition system M, and the specification φ is expressed in Computation Tree Logic (CTL). CTL enables the formal specification of critical system properties, including safety (the avoidance of undesirable states), liveness (the guarantee of system progress), reachability (the ability to attain desired goal states), and fairness (the assurance that no enabled action is indefinitely postponed).

Given a transition system and a CTL formula, model checking tools apply algorithms to exhaustively explore the system's state space, thereby enabling the formal verification of the specified properties. This makes model checking a robust and effective method for reasoning about procedural behaviors across various domains, including educational systems.

4 MCFR Framework

4.1 Overall Architecture

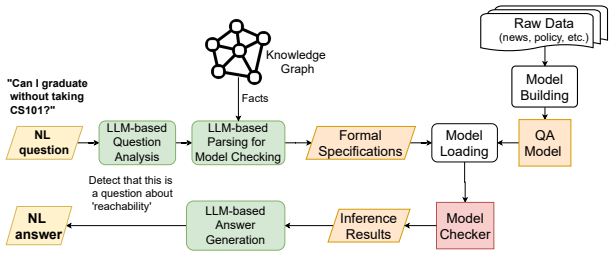


Figure 1: Overview of the MCFR framework integrating LLMs with formal verification via model checking.

Our proposed framework comprises four main components that collaboratively enable question answering grounded in formal verification. The architecture is illustrated in Figure 1.

(1) Model Building Phase. In the first stage, domain experts manually analyze raw data sources such as policy documents, institutional guidelines, and official announcements. These unstructured texts are used to construct a domain-specific QA model that encodes the system behavior, logical constraints, and domain rules required for formal reasoning. This process is performed offline and serves as the static knowledge base for the downstream inference pipeline. The detailed implementation of this QA model using a concrete model checking tool is presented in Section 4.2.

(2) Question Processing and Specification Generation. Given a user-issued NL question, the system first performs LLM-based Question Analysis to determine the intent of the query (e.g., identifying whether the question pertains to "reachability"). The system then queries the Knowledge Graph (Appendix A) to retrieve relevant facts associated with the question context. These inputs are processed by a LLM-based Parsing component that translates the question and facts into formal specifications compatible with model checking formalisms.

(3) Model Checking Inference. The generated formal specifications and the pre-built QA model are passed to the Model Loading component, which prepares them for verification. These artifacts are then submitted to a Model Checker, which performs symbolic reasoning and evaluates whether the specifications hold over the

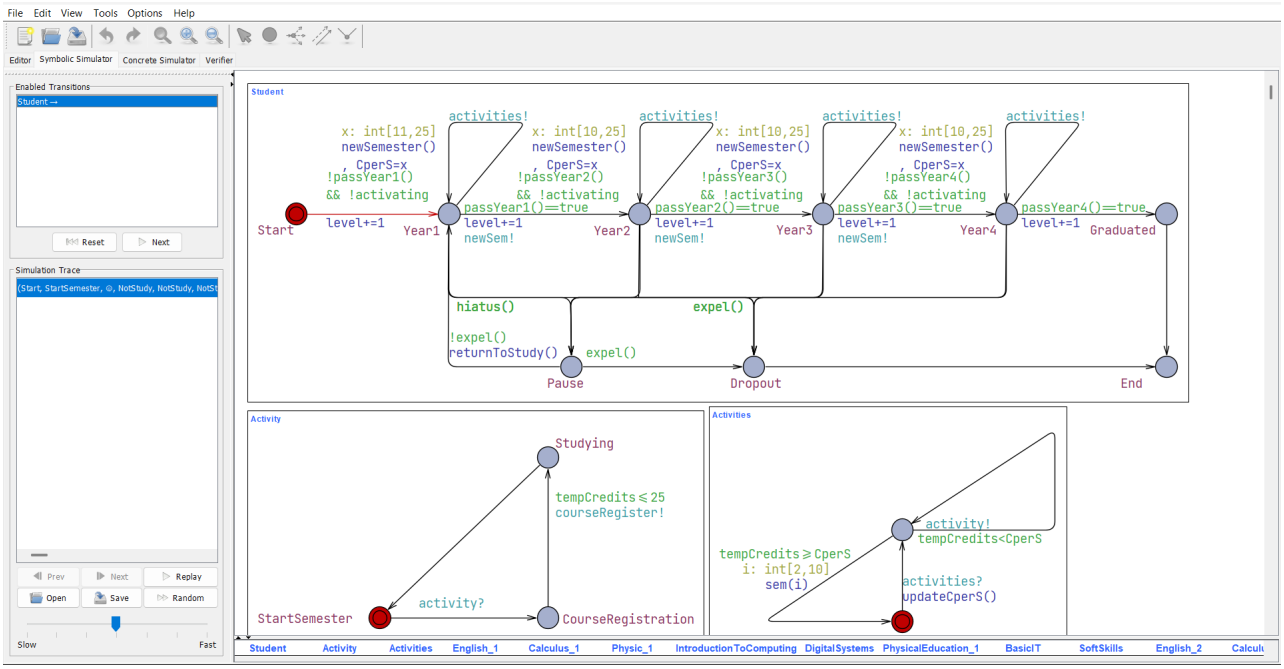


Figure 2: UPPAAL implementation of the student lifecycle model.

Natural–language Questions	UPPAAL Query	Answer	Representative Trace
Q1. Can a student graduate within eight semesters if all requirements are satisfied on time?	`E<> (Student.Graduated && time ≤ 8)`	**Yes**	`Start` → `Year1` → `Year2` → `Year3` → `Year4` → `Graduated`; `time`=8, `totalCredits`=128.
Q2. Is it possible to register for CourseX without passing its prerequisite CourseY?	`E<> (CourseX_Reg && !CourseY_Passed)`	**No**	— (no path exists; the guard `CourseY_Passed` on the CourseX transition blocks all attempts).
Q3. Can a student advance to Year 3 within three semesters?	`E<> (Student.Graduated && time ≤ 3)`	**No**	— (no path exist; completing `English_4` takes at least four semesters)
Q4. Could a student accumulate 100 credits yet fail to advance to Year 3 owing to missing social activity points?	`E<> (totalCredits ≥ 100 && Student.Year4`	**Yes**	`Start` → `Year1` → `Year2`; take electives until `totalCredits`>=100, but keep `passEnglish_4`== $false$; no transition to Year 3 enabled.

Table 1: Mapping from NL questions to UPPAAL queries, with model-checking outcomes and illustrative execution paths.

model. The outcome is returned as structured Inference Results, containing formally validated information relevant to the user query.

(4) Natural Language Answer Generation. Finally, the LLM-based Answer Generation module converts the inference results into a coherent natural language answer. This ensures that users receive not only logically sound but also linguistically accessible responses.

4.2 MCFR Implementation

Our MCFR framework follows the *NeSy Type 2* paradigm, integrating neural and symbolic components in a sequential pipeline. Specifically, Components 2 and 4 leverage the Google Gemini model[4] for natural language processing and high-level conversion tasks. In contrast, Components 3 and 4 focus on formal verification, where the domain model is constructed and executed using the UPPAAL[5] tool. This combination allows MCFR to bridge free-form natural language understanding with precise, state-based reasoning, ensuring both expressiveness and verifiability.

Building on the student lifecycle scenario in Example 1, we construct a formal model using UPPAAL, where the academic progression of a student is encoded as a network of interacting timed automata. Each component captures distinct behaviors such as course registration, semester-level coordination, and academic status transitions, all governed by temporal constraints and institutional rules (e.g., prerequisites, credit thresholds). This executable model enables simulation and formal verification of key properties such as *safety* (e.g., preventing invalid registration), *liveness* (e.g., ensuring

[4]https://deepmind.google/technologies/gemini/

[5]https://uppaal.org

eventual graduation), and *reachability* (e.g., returning from academic leave). These capabilities form the foundation for answering structured queries in our dataset.

The model uses global state variables like `totalCredits`, `time`, and Boolean flags (e.g., `passEnglish_2`, `passBasicIT`) to represent student status. Year-level progression is constrained by guard functions (e.g., `passYear1()`, `passYear2()`) that check eligibility conditions. These guards are attached to transitions to ensure policy compliance. Figure 2 illustrates the full model structure, while Table 1 presents sample verification questions along with their corresponding formal properties and execution traces.

5 EduMC-QA Dataset

To evaluate the capabilities of QA systems in performing verifiable reasoning, we construct a novel dataset, **EduMC-QA**, centered on procedural and temporal logic within the educational domain. Rather than focusing on factual recall, the questions in EduMC-QA are designed to assess whether certain properties hold over dynamic systems, such as academic workflows. Each question is annotated with one of four key verification types inspired by formal methods, which reflect core reasoning challenges commonly addressed in model checking.

Annotation Guidelines. Each question in EduMC-QA is manually assigned to one of the four verification categories based on its semantic intent and the property it seeks to verify within the academic system. The classification is grounded in definitions widely used in formal verification, with specific identification cues as shown in Table 2.

Table 2: Verification types with classification criteria

Type	Classification Criteria
Safety	◇ Ensures that nothing bad ever happens. ◇ Checks whether the system prevents invalid or undesirable actions (e.g., bypassing prerequisites).
Liveness	◇ Verifies that something good will eventually happen. ◇ Asks whether a desirable outcome can be achieved assuming all conditions are met (e.g., graduation).
Reachability	◇ Asks whether a specific state can be reached from the current state. ◇ Considers whether a goal can be achieved under certain conditions (e.g., fast-tracking to Year 3).
Fairness	◇ Evaluates whether the system behaves in a just or non-discriminatory way. ◇ Checks for unreasonable blocking despite strong compliance (e.g., enough credits but still blocked).

To ensure annotation quality and conceptual alignment, each question was independently categorized by three domain experts with backgrounds in formal methods and logical reasoning. The final label was determined by majority vote, and in the rare case of full disagreement, resolution was achieved through structured discussion and consensus.

Illustrative Classification Examples. For instance, the question Q1-"Can a student graduate within eight semesters if all requirements are satisfied on time?" is labeled as *Liveness* because it asks whether a desirable end state (graduation) will eventually occur given favorable conditions, emphasizing guaranteed progress rather than rule enforcement. Conversely, Q2-"Is it possible to register for CourseX without passing its prerequisite CourseY?" is labeled as *Safety* since it seeks to prevent an invalid action (bypassing a prerequisite), thus ensuring that no undesirable state can occur in the system.

Example Item. Each item in the dataset consists of a natural language question, a formal model, a specification, and a verification result. Additionally, two auxiliary fields are included: `context`, which contains relevant logical facts extracted from the knowledge base, and `formal_spec`, which represents the parsed formal specification and contextual information expressed in UPPAAL syntax. A simplified example is shown below:

```
{
"q": "Can a student enroll in a course without
completing its prerequisite?",
"cat": "Safety",
"context": [ "prerequisite(CourseX, CourseY)",
"student_has_not_completed(CourseY)" ],
"spec": "AG (enroll_request -> completed(prereq))",
"formal_spec": "
texttttA[] (enroll_request imply completed[prereq])",
"groundtruth": "No"
}
```

Table 3: Question distribution by verification category in EduMC-QA

Verification Property	#Q	Example Type
Safety	34	Invalid course registration
Liveness	34	Progress toward internship / graduation
Reachability	38	Path to graduation from Year 1
Fairness	23	LOA approval and return tracking
Total	129	—

Dataset Statistics. EduMC-QA comprises 129 natural language questions, each paired with a formal verification instance. The distribution across the four verification categories (Table 3) ranges from 23 to 38 items per category. The questions span diverse academic scenarios—such as enrollment, graduation, leave tracking, and internships—and vary in both linguistic and logical complexity. All items are manually annotated and validated for semantic correctness.

6 Experimentation

Building on the implementation introduced in earlier sections, we now evaluate the effectiveness of our MCFR framework by comparing its verification outcomes against responses from state-of-the-art LLMs. Using a representative academic scenario modeled in UPPAAL, we assess how well LLMs handle formal reasoning tasks and

Table 4: Representative verification questions and comparison of reasoning accuracy across MCFR and LLM baselines

Natural-language Question	Ground Truth	MCFR (Ours)	Chat GPT	Claude	Deep Seek	Observation / Reasoning Error
Q1 (Liveness). Can a student graduate within eight semesters if all requirements are satisfied on time?	**Yes**	✓	✓	✗	✓	Claude need further data to conclude
Q2 (Safety). Is it possible to register for CourseX without passing its prerequisite CourseY?	**No**	✓	✗	✗	✗	All models fail to enforce prerequisite constraints; misinterpret the question as policy-based.
Q3 (Reachability). Can a student advance to Year 3 within three semesters?	**No**	✗	✓	✓	✓	All models ignore sequential prerequisites, while completing `English_4` requires at least four semesters.
Q4 (Fairness). Could a student accumulate 100 credits yet fail to advance to Year 3 owing to missing social activity points?	**Yes**	✓	✗	✗	✗	The models do not correctly infer the academic rule that promotion to the third year requires the completion of `English_4`.

analyze their strengths and limitations in contrast to our model checking approach.

6.1 Case Study: Reasoning Comparison

To better illustrate the limitations of current LLMs in handling formal verification tasks, we present a qualitative case study comparing our neuro-symbolic system (**MCFR**) with state-of-the-art LLMs: ChatGPT, Claude, and DeepSeek. We select one representative question from each of the four verification categories in EduMC-QA—Safety, Liveness, Reachability, and Fairness—and examine whether these models can correctly answer questions that require logical inference over structured academic rules.

Each case is based on a prompt grounded in formal academic regulations, as illustrated in Example 2 (a summarized version of the full academic policy; the complete context is available at [6]). For instance, a liveness question asks whether a student can graduate within eight semesters if all requirements are satisfied on time. A safety question examines the possibility of course registration without completing prerequisites. These tasks demand reasoning over sequential transitions and conditional rules—capabilities that remain challenging for both LLMs (due to lack of verifiable grounding) and symbolic solvers (due to limited procedural expressiveness).

Example 2. Prompt Input for Evaluating LLM Reasoning
Context. Students may retake failed courses (no credit earned until passed) or improve grades by retaking passed courses with the same code (highest score used for GPA; credits unchanged). To qualify as a 2nd- or 3rd-year student, one must meet credit, language, IT, and other institutional requirements (e.g., defense education). Additionally, off-campus internships require at least 65% of total credits and must precede the graduation thesis, which also requires a completed specialized project. (See full example at: [a])
Task. Based on the context above, answer the four verification questions presented in Example 1. Each answer should be *Yes*, *No*, or *Uncertain*, and must be accompanied by a justification.

[a]https://github.com/tuanbc88-hcmut/mcfr_modelchecking_qa.git

[6]https://github.com/tuanbc88-hcmut/mcfr_modelchecking_qa.git

Table 5: Reasoning accuracy (%) by question type and models

Model	Safety (34)	Liveness (34)	Reachability (38)	Fairness (24)	Total (130)
ChatGPT-casual	79.41	82.35	63.16	66.67	73.08
ChatGPT-reason	73.53	88.24	84.21	83.33	82.31
Claude-Sonnet	44.12	50.00	44.74	12.50	40.00
DeepSeek-casual	76.47	79.41	76.32	75.00	76.92
DeepSeek-R1	52.94	55.88	50.00	37.50	50.00
MCFR(Ours)	94.12	97.06	94.74	87.50	93.85

Table 4 summarizes the correctness of model responses, using ✓ for correct and ✗ for incorrect answers. Table 5 further compares the overall reasoning accuracy across models on the EduMC-QA dataset. Detailed model responses are provided in Appendix B.

6.2 Discussion

Among LLMs, ChatGPT-reason performs best overall, particularly in liveness and reachability questions, suggesting a certain ability to reason over sequential constraints. DeepSeek-casual also delivers consistent results, while Claude-Sonnet underperforms significantly—especially on fairness—highlighting its difficulty with multi-step, policy-based reasoning. In contrast, MCFR achieves reliable reasoning via formal verification. However, it relies on a manually constructed domain model, an expert-driven and time-consuming process, which limits scalability. This points to future potential in semi-automated model construction, where LLMs assist in abstraction and experts validate critical elements. Another key challenge lies in translating NL into formal specifications; errors in this step can lead to incorrect verification outcomes even when the model is sound. Robust validation mechanisms are therefore essential to ensure end-to-end QA integrity.

NL-to-CTL Conversion Errors. A major source of error arises in converting natural language questions into formal UPPAAL queries. Current LLMs lack knowledge of the UPPAAL syntax, making it difficult to map NL descriptions into correct CTL formulas without explicit information about the target model. For accurate NL→CTL

translation, the LLM must be aware of both the UPPAAL model in use and its precise syntactic rules. Moreover, there are currently no established metrics for systematically measuring the quality of NL-to-CTL conversion, which makes it harder to benchmark and improve this step. We argue that improving the prompt—rather than extending the underlying logic model—is a more effective and immediate solution. This is because UPPAAL requires absolute syntactic correctness for verification; while logic model enhancements may improve runtime efficiency, they do not inherently boost accuracy. In contrast, prompt refinement can guide the LLM toward producing syntactically valid and semantically aligned queries.

For example, given the NL question: *"Can a student graduate within eight semesters if all requirements are satisfied on time?"*, the correct query is `E<> (Student.Graduated && time ≤ 8)`. Without prompt tuning, LLM might output `E<> (semester <= 8 && AllRequirementsMet && Student.Graduated)`, which is incorrect for two reasons: (1) the model uses the variable `time` rather than `semester`, and (2) `AllRequirementsMet` does not exist in the UPPAAL model.

QA Model Scalability and Domain Adaptation. Constructing and validating the current QA model required approximately 56 person-hours. Reproducing the model for other universities or organizations would take about 2–8 hours, depending on the extent of policy differences. The process could follow three main steps: (1) Modify institutional rules affecting student progression and graduation (up to 8 hours depending on complexity), (2) Adjust parameter values such as minimum passing grades, and (3) Conduct thorough verification testing. Beyond the academic domain, the approach can be adapted to other fields such as law or healthcare. In such cases, the time and complexity of adaptation depend heavily on the number of states, transitions, and governing rules that must be identified and modeled within each specific domain.

7 Conclusion

We introduced MCFR, a neuro-symbolic QA framework that integrates LLM-based parsing with model checking for verifiable reasoning. Evaluated on the EduMC-QA dataset, MCFR outperformed state-of-the-art LLMs in tasks requiring logical consistency and temporal verification. Our results highlight the limitations of purely neural methods and the value of hybrid approaches. Future work includes integrating Hoare Logic for local consistency checking, complementing model checking for global verification, and enabling scalable, semi-automated model development.

Acknowledgments

We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study.

References

[1] Edmund M. Clarke. 1997. Model checking. In *Foundations of Software Technology and Theoretical Computer Science*, S. Ramesh and G. Sivakumar (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 54–56.

[2] Yilun Hao, Yongchao Chen, Yang Zhang, and Chuchu Fan. 2025. Large Language Models Can Solve Real-World Planning Rigorously with Formal Verification Tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. ACL, 3434–3483. doi:10.18653/v1/2025.naacl-long.176

[3] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.

[4] Tongxuan Liu, Wenjiang Xu, Weizhe Huang, Yuting Zeng, Jiaxing Wang, Xingyu Wang, Hailong Yang, and Jing Li. 2025. Logic-of-Thought: Injecting Logic into Contexts for Full Reasoning in Large Language Models. In *Proc. of NAACL-HLT 2025 (Vol. 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). ACL, Albuquerque, New Mexico, 10168–10185. doi:10.18653/v1/2025.naacl-long.510

[5] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2025. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 666–666. doi:10.1109/ICSE55347.2025.00129

[6] Tan Tai Mai, Allie Tran, Quang-Linh Tran, An Nguyen, Hoang D. Nguyen, Tho Quan, Duc-Tien Dang-Nguyen, and Cathal Gurrin. 2025. AIQAM'25: The 2nd ACM Workshop on AI-powered Question Answering Systems for Multimedia. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)* (Dublin, Ireland). Association for Computing Machinery.

[7] Jiayuan Mao, Chuang Gan, Pushmeet Kohli, Joshua B Tenenbaum, and Jiajun Wu. [n. d.]. The Neuro-Symbolic Concept Learner: Interpreting Scenes, Words, and Sentences From Natural Supervision. In *International Conference on Learning Representations*.

[8] Katie Matton, Robert Ness, and Emre Kiciman. [n. d.]. Walk the Talk? Measuring the Faithfulness of Large Language Model Explanations. In *ICLR 2024 Workshop on Reliable and Responsible Foundation Models*.

[9] Ning Miao, Yee Whye Teh, and Tom Rainforth. [n. d.]. SelfCheck: Using LLMs to Zero-Shot Check Their Own Step-by-Step Reasoning. In *The Twelfth International Conference on Learning Representations*.

[10] Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 3806–3824. doi:10.18653/v1/2023.findings-emnlp.248

[11] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R Bowman. 2023. Language models don't always say what they think: unfaithful explanations in chain-of-thought prompting. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 74952–74965.

[12] Wenguan Wang, Yi Yang, and Fei Wu. 2025. Towards Data-And Knowledge-Driven AI: A Survey on Neuro-Symbolic Computing. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47, 2 (2025), 878–899. doi:10.1109/TPAMI.2024.3483273

Appendix: Use of Generative AI Tools

During the preparation of this paper, we used ChatGPT (developed by OpenAI) solely for the purpose of correcting grammar, spelling, and improving language clarity. The goal of this usage was to enhance readability and ensure linguistic quality, in alignment with academic writing standards. No generative AI tools were used to create research content, perform data analysis, or generate results. All content and ideas presented in the paper are solely the authors' own, and the authors take full responsibility for the final manuscript.

A EduMC-QA Knowledge Graph

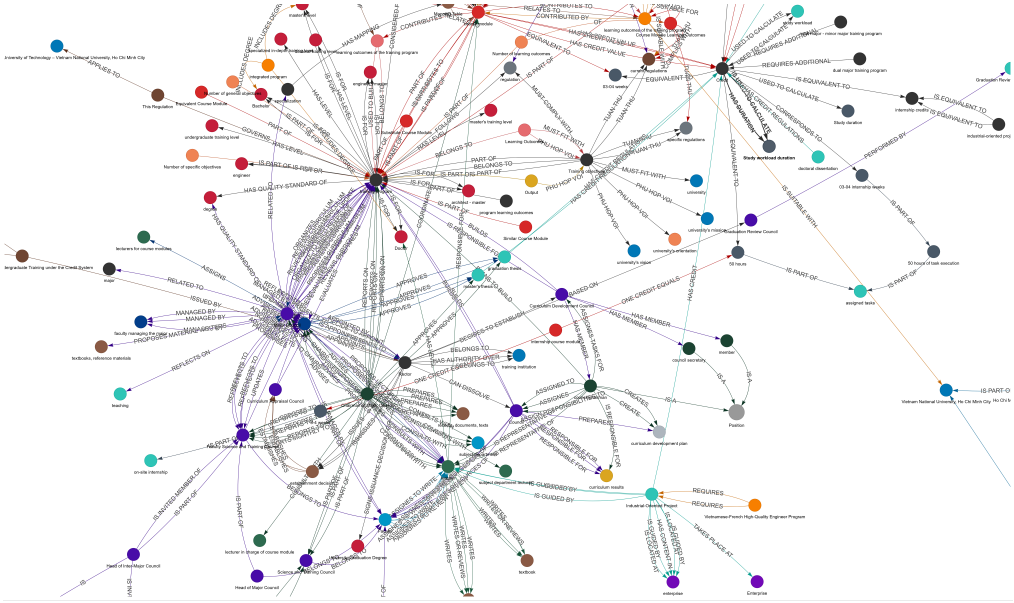


Figure 3: A subgraph of the EduMC-QA Knowledge Graph used in reasoning tasks.

B Large Language Models Response

Table 6: Comparison of Model Answers and Explanations

Question	Model	Answer & Explanation
Can a student graduate within eight semesters if all requirements are satisfied on time?	UPPAAL	**Answer:** Yes. ✓ **Explanation:** The UPPAAL query finds a valid trace. This means graduation within eight semesters is feasible.
	ChatGPT	**Answer:** Yes. ✓ **Explanation:** Assuming the program is designed for eight semesters and all requirements are fulfilled (credits, foreign language, IT skills, defense, internship, and thesis), graduation within eight semesters is achievable.
	Claude	**Answer:** Uncertain. ✗ **Explanation:** The context lacks details on credit requirements, standard load per semester, and whether eight semesters is standard. Thus, uncertainty remains.
	Deepseek	**Answer:** Yes. ✓ **Explanation:** If all requirements are met on time, there's no indication of a minimum duration beyond them. Hence, graduation within eight semesters is feasible.
Is it possible to register for CourseX without passing its prerequisite CourseY?	UPPAAL	**Answer:** No. ✓ **Explanation:** Model checking shows no valid path exists, because the guard `CourseY_Passed` on the CourseX registration transition blocks all attempts if the prerequisite is not satisfied.
	ChatGPT	**Answer:** Uncertain. ✗ **Explanation:** The context covers retakes and grade improvement but not prerequisite enforcement. Whether Course Y is required before Course X depends on broader academic rules not specified here.
	Claude	**Answer:** Uncertain. ✗ **Explanation:** The context mentions prerequisites for the graduation thesis but provides no general rules on course prerequisites or registration. It doesn't clarify if Course Y must be passed before Course X.
	Deepseek	**Answer:** Uncertain. ✗ **Explanation:** The context doesn't specify rules on course prerequisites or whether they can be bypassed. It mentions retakes and credit accumulation but lacks enough information for a definitive answer.
Can a student advance to Year 3 within three semesters?	UPPAAL	**Answer:** No. ✓ **Explanation:** The query has no valid path, because completing required courses like `English_4` takes at least four semesters.
	ChatGPT	**Answer:** Yes. ✗ **Explanation:** If a student takes heavy course loads or summer semesters and manages to earn at least 56 credits and complete all additional requirements in 3 semesters, they can advance to Year 3.

	Claude	**Answer:** Yes. ✗ **Explanation:** The context does not specify any minimum time requirements or restrictions that would prevent a student from advancing to Year 3 in 3 semesters, so advancement is possible if all academic requirements are fulfilled.
	Deepseek	**Answer:** Uncertain. ✗ **Explanation:** The context does not provide enough information to determine whether all 3rd-year requirements can be met in 3 semesters.
Could a student accumulate 100 credits yet fail to advance to Year 3 owing to missing social activity points?	UPPAAL	**Answer:** Yes. ✓ **Explanation:** A student can take electives until `totalCredits>=100`, but keep `passEnglish_4==` $false$; the transition to Year 3 is blocked.
	ChatGPT	**Answer:** No. ✗ **Explanation:** The year advancement requirements for Year 3 include credit, language, IT skills, and defense education. Since social activity points aren't listed, lacking them won't block progression.
	Claude	**Answer:** No. ✗ **Explanation:** The context states that Year 3 status requires ≥ 56 credits, foreign language, IT skills, and defense education. Social activity points aren't mentioned, so the listed requirements are sufficient for advancement.
	Deepseek	**Answer:** No. ✗ **Explanation:** The requirements for Year 3 include ≥ 56 credits, English 1-3, IT skills, and defense education. Social activity points aren't mentioned, so lacking them wouldn't prevent progression if academic conditions are met.
Is it always true that if a student has not passed English 1, then they have not met the second-year foreign language requirement?	UPPAAL	**Answer:** No. ✓ **Explanation:** Students having IELTS certificates with score above 6 automatically meet all foreign language requirements.
	ChatGPT	**Answer:** Yes. ✗ **Explanation:** According to the regulation, "Requirements for 2nd-Year Student Status" include "Passed English 1" as part of the foreign language standard. Therefore, any student who hasn't passed English 1 clearly does not meet this requirement.
	Claude	**Answer:** Yes. ✗ **Explanation:** According to the regulation, passing English 1 is mandatory for meeting the 2nd-year language standard. As it is the foundation of the sequence, failing it also blocks students from fulfilling the 3rd- and 4th-year standards requiring English 1–3 and 1–4.
	Deepseek	**Answer:** Yes. ✗ **Explanation:** According to the regulation, passing English 1 is required for the 2nd-year language standard. Without it, students cannot meet the requirements for Years 2 to 4.
Can I register for the specialized project if I haven't completed English 2?	UPPAAL	**Answer:** Yes. ✓ **Explanation:** English 2 is not a requirement for the specialized project.
	ChatGPT	**Answer:** Yes. ✓ **Explanation:** The context does not list English 2 as a requirement for the specialized project.
	Claude	**Answer:** Uncertain. ✗ **Explanation:** The context does not specify any prerequisites for registering for the specialized project.
	Deepseek	**Answer:** No. ✗ **Explanation:** According to the regulations, completing English 2 is an absolute prerequisite before registering for specialized project courses. No exemptions from this requirement are provided in the current rules.
Can I take the internship if I am missing exactly one foundational course?	UPPAAL	**Answer:** Yes ✓ **Explanation:** Foundational courses are not required to be all complete in take internship.
	ChatGPT	**Answer:** Uncertain ✗ **Explanation:** The context does not mention a specific requirement to complete all foundational courses.
	Claude	**Answer:** Uncertain ✗ **Explanation:** Without knowing the total program credits or how many credits the missing foundational course represents, it is impossible to determine if the student would still meet the 65 % threshold required for internship eligibility.
	Deepseek	**Answer:** No. ✗ **Explanation:** Students must complete every single foundation course without exception to qualify for internships, as the regulations strictly enforce this prerequisite with no flexibility.
Is it always true that if a student completes English 1–3, they are automatically recognized as having met the Year-3 foreign language requirement?	UPPAAL	**Answer:** No. ✓ **Explanation:** Year-3 foreign language requirement is English 1, 2, 3, and 4.
	ChatGPT	**Answer:** No. ✓ **Explanation:** The context clearly states that the Year-3 foreign language requirement is English 1, 2, 3, and 4. Therefore, completing only English 1–3 is not sufficient to meet the requirement.
	Claude	**Answer:** No. ✓ **Explanation:** The context clearly states that the 4th-year foreign language requirement for 3rd-year student status requires "Passed English 1, 2, 3, and 4." A student who has only completed English 1-3 is missing English 4, which is explicitly required. Therefore, completing only English 1-3 does not automatically satisfy the Year-3 foreign language requirement.
	Deepseek	**Answer:** Yes. ✗ **Explanation:** The document stipulates that completing English 1-3 fully satisfies the third-year foreign language requirement. No alternative provisions or additional conditions are mentioned in these regulations.